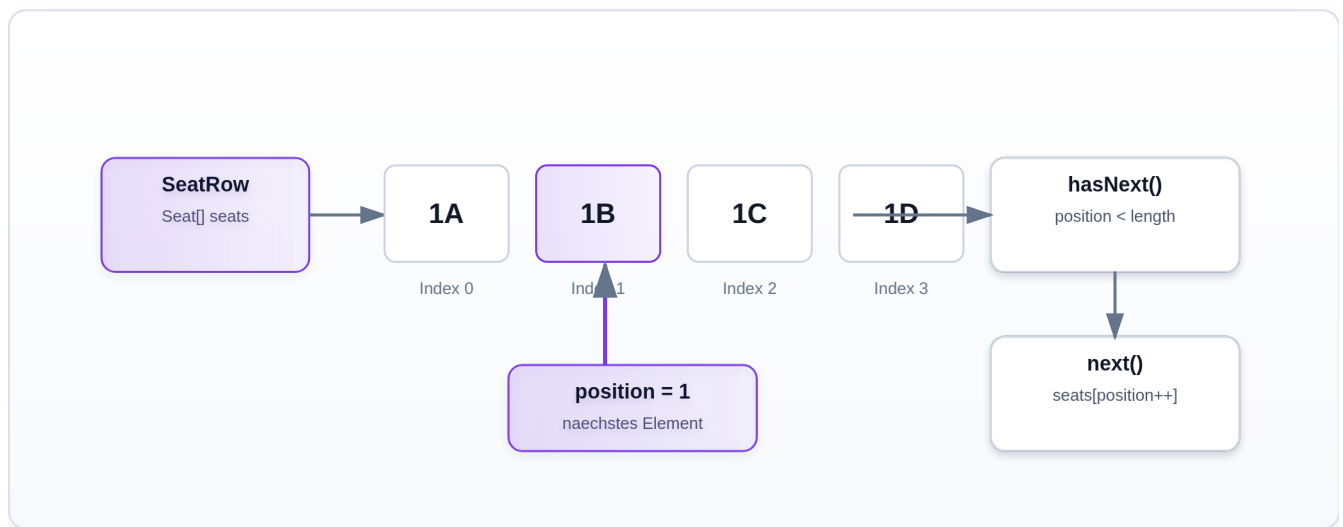


Aufgabe 03: Eigener Iterator fuer ein Array

Erklaerung mit Ablaufdiagramm, zentralem Iterator-Zustand und Bezug zur konkreten Java-Implementierung.

● Professionelles Ablaufdiagramm



■ Grundidee

`SeatRow` speichert Sitzplaetze in einem Array:

```
private final Seat[] seats;
```

Der Iterator `SeatRowIterator` soll diese Sitzplaetze nacheinander liefern.

Bei `SeatRow.sample()` ist die erwartete Reihenfolge:

```
1A
1B
1C
1D
```

Welchen Zustand braucht der Iterator?

Ein Array hat feste Positionen:

```
Index 0 -> 1A  
Index 1 -> 1B  
Index 2 -> 1C  
Index 3 -> 1D
```

Der Iterator muss sich merken, welche Position als naechstes geliefert werden soll.

Dafuer gibt es das Feld:

```
private int position;
```

`position` startet automatisch bei `0`, weil `int`-Felder in Java den Standardwert `0` haben.

Konstruktor

Der Konstruktor bekommt das Array von `SeatRow`:

```
public SeatRowIterator(Seat[] seats) {  
    this.seats = seats;  
}
```

Der Iterator speichert dieses Array, damit er spaeter mit `position` darauf zugreifen kann.

Die Rolle von hasNext()

`hasNext()` prueft, ob die aktuelle Position noch innerhalb des Arrays liegt:

```
@Override  
public boolean hasNext() {  
    return position < seats.length;  
}
```

Solange `position` kleiner als `seats.length` ist, gibt es noch ein Element.

Beispiel bei vier Sitzplaetzen:

```
position = 0 -> gueltig
position = 1 -> gueltig
position = 2 -> gueltig
position = 3 -> gueltig
position = 4 -> nicht mehr gueltig
```

Darum ist die Grenze `position < seats.length` und nicht `position <= seats.length`.

Die Rolle von next()

`next()` liefert den aktuellen Sitzplatz und bewegt den Iterator danach weiter:

```
@Override
public Seat next() {
    if (!hasNext()) {
        throw new NoSuchElementException();
    }

    return seats[position++];
}
```

Zuerst wird geprueft, ob noch ein Element vorhanden ist.

Wenn nicht, wird eine `NoSuchElementException` geworfen. Das gehoert zum Vertrag von `Iterator.next()`.

Wenn ein Element vorhanden ist, passiert in dieser Zeile alles Wichtige:

```
return seats[position++];
```

Das bedeutet:

1. Lies das Element an der aktuellen Position.
2. Gib dieses Element zurueck.
3. Erhoehe `position` danach um `1`.

`position++` ist ein Post-Increment. Der alte Wert wird zuerst verwendet, danach wird er erhoeht.

Ablauf am Beispiel

Am Anfang:

```
position = 0
```

Erster Aufruf von `next()`:

```
seats[0] -> 1A  
position wird 1
```

Zweiter Aufruf:

```
seats[1] -> 1B  
position wird 2
```

Dritter Aufruf:

```
seats[2] -> 1C  
position wird 3
```

Vierter Aufruf:

```
seats[3] -> 1D  
position wird 4
```

Jetzt ist `position == seats.length`. `hasNext()` liefert `false`.

Zusammenspiel mit SeatRow

`SeatRow` implementiert `Iterable<Seat>` und erzeugt den Iterator:

```
@Override  
public Iterator<Seat> iterator() {  
    return new SeatRowIterator(seats);  
}
```

Dadurch funktioniert:

```
for (Seat seat : SeatRow.sample()) {  
    System.out.println(seat);  
}
```

Die `for-each`-Schleife ruft intern `iterator()`, `hasNext()` und `next()` auf.

Zusammenfassung

Der Iterator braucht fuer ein Array nur zwei Dinge:

- Das Array selbst.
- Einen Index auf das naechste Element.

`hasNext()` prueft die Grenze des Arrays.

`next()` liefert das aktuelle Element und erhoehrt danach den Index.

So wird aus einem Array eine sauber durchlaufbare Iterator-Struktur.